

CS5330 Computer Vision – Project 1

Image Manipulation and Video Filtering

Shriman Raghav Srinivasan

Shyam Srinivasan

May 2026

Project Overview

For this project we built a live video filtering program in C++ using OpenCV. The idea is that you run the program, point your webcam at something, and then use different keys to switch between filters in real time. We implemented filters for greyscale (two versions), sepia tone, Gaussian blur (two implementations at different speeds), Sobel edge detection, gradient magnitude, and a blur-and-quantize effect. We also integrated a face detector using a Haar cascade and a depth estimation model called Depth Anything V2, which runs through the ONNX runtime. On top of those, we added three custom effects: color negative, emboss, and depth-based fog.

All the pixel manipulation functions are written from scratch in `filter.cpp`. The main loop lives in `vidDisplay.cpp` and just reads a frame, applies whatever filter is currently selected, and shows it. We split the work: Shriman handled Tasks 4 through 9 (`filters.cpp`) and Shyam handled the rest.

Required Images and Results

Required Image 1 – OpenCV Greyscale (Task 3)



(a) Original color frame



(b) `cvtColor` greyscale output

Figure 1: Task 3: Standard greyscale using `cv::COLOR_BGR2GRAY`. According to the OpenCV docs, it uses the formula $Y = 0.299R + 0.587G + 0.114B$, which comes from the ITU-R BT.601 standard. Green gets the most weight because the human eye is most sensitive to green light.

Required Image 2 – Custom Greyscale (Task 4)



(a) OpenCV greyscale ($0.299R + 0.587G + 0.114B$)



(b) Custom greyscale ($0.15R + 0.35G + 0.50B$)

Figure 2: Task 4: Custom greyscale with a blue-channel emphasis. We used weights $0.15R + 0.35G + 0.50B$ so blue is weighted about 4 times more than in the standard formula. This makes warm-colored (red/orange) regions look darker and cool-colored (blue) regions look brighter compared to the OpenCV version. All three output channels are set to the same value so it is actually greyscale. Pixels are accessed using row pointers (`ptr<Vec3b>`).

Required Image 3 – Sepia Tone (Task 5)



(a) Original color frame



(b) Sepia filter output

Figure 3: Task 5: Sepia tone filter. Each output channel is a weighted combination of all three input channels: $B' = 0.272R + 0.534G + 0.131B$, $G' = 0.349R + 0.686G + 0.168B$, $R' = 0.393R + 0.769G + 0.189B$. To make sure we always use the original values, we read all three channels into float variables before writing anything. That way no output channel can accidentally influence another. We also clamp values to 255 since the red and green rows of coefficients sum to more than 1.

Required Image 4 – 5x5 Gaussian Blur (Task 6)



(a) Original color frame



(b) After `blur5x5_2`

Figure 4: Task 6: 5x5 Gaussian blur applied to each color channel separately. The kernel used is $[1\ 2\ 4\ 2\ 1; 2\ 4\ 8\ 4\ 2; 4\ 8\ 16\ 8\ 4; 2\ 4\ 8\ 4\ 2; 1\ 2\ 4\ 2\ 1]$ with a sum of 100.

Timing Comparison (960x720 image, average of 5 runs)

Function	Avg. time	Method
<code>blur5x5_1</code>	16.01 ms	Nested loop with <code>.at<></code>
<code>blur5x5_2</code>	3.81 ms	Separable passes and row pointers

Table 1: `blur5x5_2` is about 4x faster for two reasons. First, we split the 2D filter into a horizontal pass followed by a vertical pass using the 1D kernel $[1\ 2\ 4\ 2\ 1]$. This cuts operations per pixel from 25 down to 10. Second, we access pixels through row pointers (`ptr<Vec3b>`) instead of `.at<>`, which avoids bounds-checking on every access.

Required Images 5 – Sobel Filters and Gradient Magnitude (Tasks 7 and 8)



(a) Original



(b) Sobel X (absolute value shown)



(c) Sobel Y (absolute value shown)



(d) Gradient magnitude

Figure 5: Tasks 7 and 8: Sobel edge detection and gradient magnitude. Sobel X picks up vertical edges (left-to-right intensity changes) using the separable filters $[-1 \ 0 \ 1]$ horizontal then $[1 \ 2 \ 1]$ vertical, with positive values meaning a right-going gradient. Sobel Y picks up horizontal edges using $[1 \ 2 \ 1]$ horizontal then $[1 \ 0 \ -1]$ vertical, with positive values meaning an upward gradient. Both functions output `CV_16SC3` (signed short) because the values can be negative and can go up to 1020 in magnitude. We keep a separate variable for display and use `convertScaleAbs` only for that, not for the filter output itself. The magnitude image combines both directions: $M = \sqrt{s_x^2 + s_y^2}$ per channel, clamped to 255.

Required Image 6 – Blur and Quantize (Task 9)



(a) Original color frame



(b) `blurQuantize` with 10 levels

Figure 6: Task 9: Blur then quantize. We first run `blur5x5.2` to smooth the image, then snap each channel to the nearest bucket value: $bucket = 255/levels$, $x_t = \lfloor x/bucket \rfloor$, $x_f = x_t \times bucket$. With 10 levels per channel the image can have at most $10^3 = 1000$ distinct colors, which gives a cartoon or poster look. Blurring first helps because without it the quantization looks noisy.

Required Image 7 – Face Detection (Task 10)

[Screenshot to be added by Shyam: press `f` in `vidDisplay`, then `s` to save.]

Required Images 8 and 9 – Depth Estimation (Task 11)

[Screenshots to be added by Shyam: press `d` for the depth map and `c` for the depth-defocus creative filter.]

Required Images 10 to 12 – Custom Effects (Task 12)

[Screenshots to be added by Shyam: press `n`, `e`, and `o` respectively.]

We chose three effects that each meet one of the requirements:

Color Negative (n, pixel-wise): Inverts every channel value with $dst[c] = 255 - src[c]$. No neighbors needed.

Emboss (e, area-based): Takes the signed Sobel X and Y outputs from Tasks 7 and 8, computes their dot product with a light direction of (0.7071, 0.7071), averages across the three channels, then adds 128 so flat areas map to mid-grey.

Depth Fog (o, uses depth): Uses the DA2 depth map to add exponential fog. The formula is $fog = 1 - e^{-k \cdot d}$ where $d = 1 - depth/255$. Close objects stay clear and far objects fade toward a grey fog color.

Extensions

We did not pursue formal extensions beyond the required tasks. The depth fog filter (Task 12c) builds directly on Task 11 using a physically-based model for atmospheric scattering rather than a simple depth threshold, so it could count as a small extension of that task.

Reflection

- Working at the pixel level with `cv::Mat` made us realize that images are just arrays of numbers, and most of what makes a filter look good is a careful choice of weights or thresholds applied to those numbers.
- Writing two versions of the blur filter and timing them taught us that algorithmic decisions matter more than we expected. Switching from `.at<>` to row pointers and using separable 1D passes instead of a full 2D kernel gave a 4x speedup on the same hardware.
- The Sobel filter was initially confusing because Sobel X detects horizontal changes but shows vertical edges in the output. Reasoning through that helped us understand the relationship between the direction of a gradient and the orientation of edges.
- Using signed 16-bit output for the Sobel functions was something we had not thought about before. Without it, negative gradient values would be clipped silently and the results would be wrong.
- Integrating Depth Anything V2 through the ONNX runtime showed us how modern deep networks can fit into a classical CV pipeline. Having per-pixel depth information opens up filter ideas that are not possible with just the RGB values.
- Face detection with a Haar cascade was fast and surprisingly easy to wire in, which made us curious about how much more accurate but slower methods like deep learning detectors compare in practice.
- Overall this project changed how we look at filters in tools like Photoshop or Instagram. What seemed like magic before is now a pretty understandable sequence of per-pixel math operations.

Acknowledgements

- OpenCV documentation (<https://docs.opencv.org/4.x/>) for the `cvtColor` channel weights and pixel access patterns.
- Shriman Raghav Srinivasan implemented Tasks 4 through 9, including the custom greyscale, sepia tone, both blur implementations, Sobel X and Y filters, gradient magnitude, and blur-and-quantize functions in `filter.cpp` and `filter.h`.
- Shyam Srinivasan implemented Tasks 1 through 3 and Tasks 10 through 12 (face detection, Depth Anything V2, and the three custom effects).
- We used Claude (Anthropic) as an AI assistant to help set up the initial code structure and for debugging during development.